ProSEIT

PROSEIT ANGULAR EXPERIENCE PLATFORM

PAEP

Clean-Room Angular Design System & Enterprise Application Template Specification

DESIGN SYSTEM • ENTERPRISE SHELL • TYPED CONTRACTS • FRAPPE ADAPTER • ANALYTICS & REPORTING • ACCESSIBILITY • I18N • OFFLINE/PWA • SECURITY • PORTABILITY • CLEAN-ROOM PROVENANCE

Field	Controlled Value
Document ID	PROSEIT-STD-PMRA-AF-006 | PMRA-AF-06
Version	1.1 | 14 September 2026
Status	Review-ready amended controlled child specification of PMRA-AF Version 2.0
Solutions Architect & Main Product Owner	Dr Kamulegeya Grace B (PhD)
Developing Organisation	PROSEIT ORGANISATION OF SOFTWARE PROFESSIONALS LTD
Governing Architecture	PMRA-AF Version 2.0, 25 August 2026
Technology Baseline	Angular; ProSEIT-owned PAEP source/design system; @proseit/frappe-sdk; ERPNext/Frappe domain APIs

CONTROLLED PLATFORM SPECIFICATION

Defines the shared clean-room frontend platform from which authorised ProSEIT product experiences shall be composed. Product-specific screens, workflows and domain logic remain in their respective product specifications.

Document Control

Field	Controlled Value
Document title	ProSEIT Angular Experience Platform (PAEP) - Clean-Room Angular Design System & Enterprise Application Template Specification
Document ID	PROSEIT-STD-PMRA-AF-006 / PMRA-AF-06
Version	1.1
Status	Review-ready amended controlled child specification
Effective date	Upon Product Owner / Architecture approval
Parent architecture	PMRA-AF Version 2.0 - PROSEIT-STD-PMRA-AF-001
Solutions Architect	Dr Kamulegeya Grace B (PhD)
Main Product Owner	Dr Kamulegeya Grace B (PhD)
Developing organisation	PROSEIT ORGANISATION OF SOFTWARE PROFESSIONALS LTD
Primary backend integration	ERPNext/Frappe through versioned PMRA domain APIs and @proseit/frappe-sdk
Intended reuse	Manufacturing, agriculture, BuildOS and future authorised ProSEIT product families
Commercial-template posture	No proprietary UI-template runtime dependency or copied proprietary template implementation in the PAEP baseline

Revision History

Version	Date	Disposition	Principal Change
1.0	3 Sep 2026	Review candidate	Packages the PAEP concepts defined in PMRA-AF v2.0 into the controlled PMRA-AF-06 child specification; adds explicit workspace/repository boundaries, runtime configuration, security, offline/PWA, test, release, adoption and provenance controls.
1.1	14 Sep 2026	Review candidate	Strengthens modular dependency rules, Angular engineering profile, adaptive UX, security, typed runtime configuration, deployment portability and product-neutral contracts; adds PAEP rich reporting/analytics architecture and release gates.

Approval and Decision Rights

Decision domain	Primary authority	Required supporting review
PAEP architecture, boundaries and platform exceptions	Solutions Architect	PMRA Architecture Review Board
Product experience priorities and release acceptance	Main Product Owner	Affected product leads and Frontend Platform Council

Decision domain	Primary authority	Required supporting review
Design system, accessibility and UX standards	Frontend Platform Council / Quality Lead	Product UX, accessibility and QA evidence
Security, identity, tenancy and frontend trust boundaries	Security and Assurance Authority	Threat model, secure design and test evidence
Open-source dependencies, licences and provenance	Engineering Governance / IP owner	Licence review, SBOM and provenance evidence
Frappe SDK/API compatibility	Platform Integration Lead	Backend/domain owners and contract-test evidence
PAEP release and breaking changes	Solutions Architect + Main Product Owner	Compatibility matrix, migration notes and release gates

CONTROLLED-DOCUMENT NOTICE

PAEP is a software architecture and engineering specification. It does not transfer or remove third-party open-source licence obligations. Angular, Frappe, ERPNext and approved libraries retain their respective licences. The controlled objective is ProSEIT ownership of the application shell, design system, domain component implementation and platform integration code, with provenance and dependency obligations retained as evidence.

NORMATIVE LANGUAGE

“Shall” and “must” denote mandatory PAEP controls; “should” denotes a recommended control requiring documented justification when not applied; “may” denotes an optional implementation choice. A product may tailor visual accents and domain components, but shall not bypass PAEP security, accessibility, provenance, API-contract or release controls.

Table of Contents

1. Executive Summary
2. Purpose, Scope and Non-goals
3. PAEP Architecture Principles
4. PAEP Position in the ProSEIT Architecture
5. Logical Architecture and Workspace Structure
6. Design Foundations and Tokens
7. Enterprise Application Shell
8. Reusable Component Architecture
9. Domain Component Kits
10. @proseit/frappe-sdk and Backend Integration
11. Identity, Tenancy, Permissions and Edition Gating
12. Routing, Runtime Configuration and Feature Composition
13. Realtime, Files, Notifications and Background Work
14. Offline/PWA and Low-Bandwidth Architecture
15. Internationalisation and Localisation
16. Accessibility and Inclusive Interaction
17. Security, Privacy and Frontend Trust Boundaries
18. Performance, Reliability and Observability
19. Clean-Room Provenance and Licence Governance
20. Testing, Quality Attribute Contracts and Release Gates
21. PAEP Showcase / UI Laboratory and Developer Experience
22. Versioning, Packaging and Release Management
23. Product Adoption and Migration
24. Implementation Roadmap
25. Architecture Acceptance and Evidence
26. Principal Risks and Controls
27. Rich Reporting and Analytics Architecture

Appendix A. PAEP Repository Reference Structure
Appendix B. PAEP Clean-Room Definition of Done
Appendix C. Component Acceptance Checklist
Appendix D. Product Adoption Checklist
Appendix G. Angular Engineering Profile

Appendix H. Reporting and Analytics Acceptance Checklist

Appendix E. Acronyms
Appendix F. References

1. Executive Summary

The ProSEIT Angular Experience Platform (PAEP) is the shared frontend platform for the ProSEIT product family. PMRA-AF Version 2.0 establishes PAEP to replace the earlier Ultima/PrimeNG-governed template posture with a ProSEIT-owned, clean-room Angular platform that can deliver comparable enterprise breadth, consistency and polish without a proprietary commercial-template runtime or redistribution dependency [1].

PAEP is not a generic website theme. It is an enterprise application platform comprising design foundations, reusable interaction components, an application shell, product-neutral platform integrations, a typed Frappe SDK, accessibility and localisation controls, offline/PWA patterns, testing and release governance, and evidence supporting clean-room provenance.

ARCHITECTURE THESIS

One controlled Angular experience platform; multiple independently branded product experiences; backend-authoritative permissions and domain logic; typed APIs instead of arbitrary DocType coupling; clean-room implementation with retained provenance; and measurable frontend quality from component to release.

1.1 Strategic outcomes

- Reuse one enterprise shell, design system and integration foundation across manufacturing, agriculture, construction and future ProSEIT products.
- Preserve product-specific identity through controlled tokens, domain kits, navigation and route composition rather than by forking the platform.
- Make frontend access safe for African operating conditions through responsive design, PWA/offline capability, low-bandwidth patterns and explicit synchronization state.
- Provide stable ERPNext/Frappe integration through @proseit/frappe-sdk and versioned PMRA domain APIs rather than direct persistence coupling.
- Treat accessibility, internationalisation, security, performance, provenance and upgradeability as release gates rather than optional finishing work.
- Retain a demonstrable evidence trail showing that PAEP is independently designed and implemented and that open-source dependency obligations are known and managed.

1.2 Revisions introduced in this standalone specification

Area	PMRA-AF v2.0 baseline	Revision / clarification in PAEP 1.0
Platform scope	Design system, enterprise shell, domain kits and Frappe SDK	Adds explicit repository/workspace boundaries, package roles, runtime configuration and product composition rules.
Clean-room	No copied proprietary template source/assets; record provenance and licences	Adds a gated provenance workflow, evidence register, dependency approval and release-level attestation.
Backend integration	Use stable domain APIs and @proseit/frappe-sdk	Defines typed client contracts, error model, concurrency/idempotency expectations and contract tests.
Offline/PWA	Responsive/PWA/low-bandwidth support	Defines local/queued/synced/conflicted states, retry/reconciliation rules and offline security constraints.
Accessibility	WCAG 2.2 Level AA target	Adds keyboard/focus, semantics, error recovery, screen-reader and manual test gates.
Release governance	Component tests, visual regression, SBOM and showcase	Adds versioning, compatibility matrix, migration notes, performance budgets and controlled product adoption.

1.3 Amendments introduced in Version 1.1

Area	Version 1.1 controlled amendment

Modularity	Adds explicit package dependency direction, published entry-point rules and CI-enforced architecture boundaries.
Angular engineering	Adds standalone-first composition, signal-oriented state, zoneless compatibility, lazy/deferred loading, strict typing and AOT production rules.
Portability	Separates product-neutral PAEP contracts from the ERPNext/Frappe adapter and defines deploy-once/runtime-configure deployment profiles.
Security	Makes production CSP, Trusted Types where supported, AOT, browser-storage controls and secure offline-data lifecycle explicit.
Responsive UX	Adds measurable reflow/adaptive interaction rules, enterprise table adaptation and representative-user usability evidence.
Reporting & analytics	Adds governed operational and analytical reporting capable of rich interactive dashboards, drill-through, semantic measures, exports and optional analytical stores without browser/database coupling.

2. Purpose, Scope and Non-goals

2.1 Purpose

This document is the controlled child specification PMRA-AF-06 identified in the PMRA-AF Version 2.0 downstream document set [1]. It constrains the design, implementation, packaging, verification and reuse of PAEP across authorised ProSEIT products.

2.2 In scope

- Angular workspace and package boundaries; application shell; design tokens; reusable components; product composition.
- Responsive web, tablet/mobile web and PWA behaviour, including bounded offline workflows.
- Internationalisation and RTL support; accessibility and inclusive interaction.
- Typed Frappe/ERPNext integration, authentication/session, workflows, files, realtime, errors and observability.
- Frontend security, privacy, tenant/edition presentation, release configuration and feature composition.
- Clean-room provenance, dependency governance, SBOM, test evidence, compatibility and release controls.
- PAEP showcase/UI laboratory, developer documentation and product migration/adoption.
- Product-neutral PAEP contracts, backend adapters and deployment-portability controls.
- Rich reporting and analytics experiences, semantic report metadata, dashboards, exports and approved operational/analytical query paths.
- Angular engineering profile, state-management discipline, strict build/security posture and dependency-boundary enforcement.
- UX interaction principles and representative-user usability evidence for critical workflows.

2.3 Explicit non-goals

- PAEP shall not implement authoritative accounting, tax, fiscal, safety, entitlement or domain validation that belongs in backend/domain services.
- PAEP is not a clone of Ultima, PrimeNG, SAP Fiori or any other proprietary or distinctive commercial template implementation.
- PAEP does not require every product to have identical screens; domain workflows and information architecture remain product-specific.
- PAEP does not remove licence obligations for Angular, Frappe, ERPNext or approved open-source dependencies.
- PAEP does not mandate micro-frontends. A modular Angular workspace is preferred until independently deployable frontend boundaries have a justified architecture driver.
- PAEP shall not turn the browser into a BI/database client. Operational databases, analytics stores and unrestricted query languages shall not be exposed directly to frontend code; all reporting queries remain governed through authorised APIs and semantic/query services.

3. PAEP Architecture Principles

Principle	Mandatory interpretation
Clean-room ownership	PAEP source, layouts, tokens and component implementations shall be independently designed. Proprietary template source, theme files, assets or copied distinctive implementation artefacts shall not enter the baseline.

Principle	Mandatory interpretation
Backend authority	Permissions, entitlements, tax/fiscal decisions, workflow validity and consequential domain state changes remain authoritative in backend/domain services.
Contracts before persistence	Frontend clients consume versioned domain APIs/events through approved SDK clients; arbitrary persistence-model coupling is prohibited except for explicitly approved low-risk platform metadata.
Product family reuse	Shared shell, components and integrations are built once; products extend through tokens, routes, domain kits and configuration rather than forks.
Accessible by default	Components must expose semantic roles, keyboard/focus behaviour, errors/status and adequate contrast as part of their definition of done.
Offline truth is explicit	The UI must disclose whether data is local, queued, synchronized, conflicted or failed. Silent data loss or ambiguous sync state is unacceptable.
Secure configuration	Environment-specific endpoints, identity settings, feature flags and edition metadata are runtime/configuration concerns and must not require source forks.
Measurable quality	Performance, accessibility, security, reliability and upgradeability targets are testable and evidence-backed.
Progressive architecture	Prefer one governed Angular workspace and shared packages; extract separate deployables only for independently justified scale, security, release or ownership boundaries.

Additional principle	Mandatory interpretation
Contract/adapter separation	Reusable PAEP UI and domain packages depend on product-neutral typed contracts. ERPNext/Frappe remains the strategic backend through an approved @proseit/frappe-sdk adapter that implements those contracts.
Dependency direction	Dependencies flow from apps/domain/enterprise toward reusable components, contracts and foundations through published entry points. Circular and internal-path coupling is prohibited.
State locality	Keep state as close as practical to the owning component/feature; backend/server state remains authoritative. Application-wide mutable stores require a documented architecture driver.
Adaptive UX	Responsive design means task-preserving adaptation, not only fluid resizing. Critical actions, context and meaning remain usable across supported viewport, touch, zoom/reflow and locale conditions.
Portable deployment	Environment-specific endpoints, identity, telemetry and product settings are runtime configuration. Product source forks for deployment environments are prohibited.
Governed analytics	Reports use controlled measures, dimensions, tenant/row security and bounded query paths. Heavy historical analytics are isolated from operational transaction workloads where required.
Evidence-based UX	Accessibility compliance is necessary but not sufficient; critical workflows also require usability evidence from representative users or approved equivalent evaluation.

4. PAEP Position in the ProSEIT Architecture

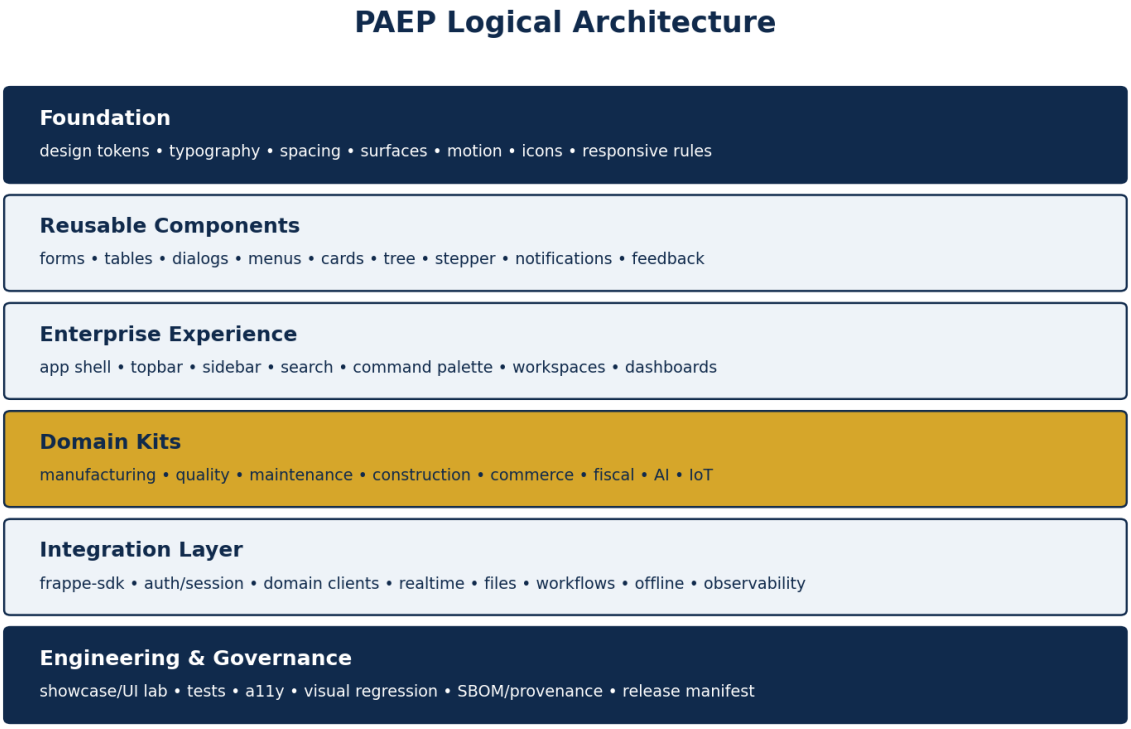
PAEP is the user-experience layer above product-specific domain capabilities and PMRA-AF shared services. It does not replace product architecture or backend services; it standardises how ProSEIT products present, compose and integrate those capabilities.

Figure 1. PAEP position in the ProSEIT platform architecture.

4.1 Responsibility boundary

Concern	PAEP responsibility	Backend / domain responsibility
Identity	Session presentation, login flows, renewal handling, identity context	Credential validation, token/session authority, SSO policy, account lifecycle
Permissions	Hide/disable/label unavailable actions; explain access	Authoritative RBAC/ABAC, tenant scope and permission checks
Editions	Present entitled routes/features and upgrade prompts	Authoritative entitlements, limits, subscriptions and resource enforcement
Workflow	Render state, permitted action affordances, approvals UI	Validate transitions, segregation of duties, write state, audit evidence
Fiscal/financial	Display calculated states and collect approved inputs	Tax/fiscal calculation, posting, reconciliation, statutory decisions
AI/IoT	Provide workspaces, approvals, telemetry/agent presentation	Model/tool governance, telemetry truth, safety controls and authoritative actions

5. Logical Architecture and Workspace Structure



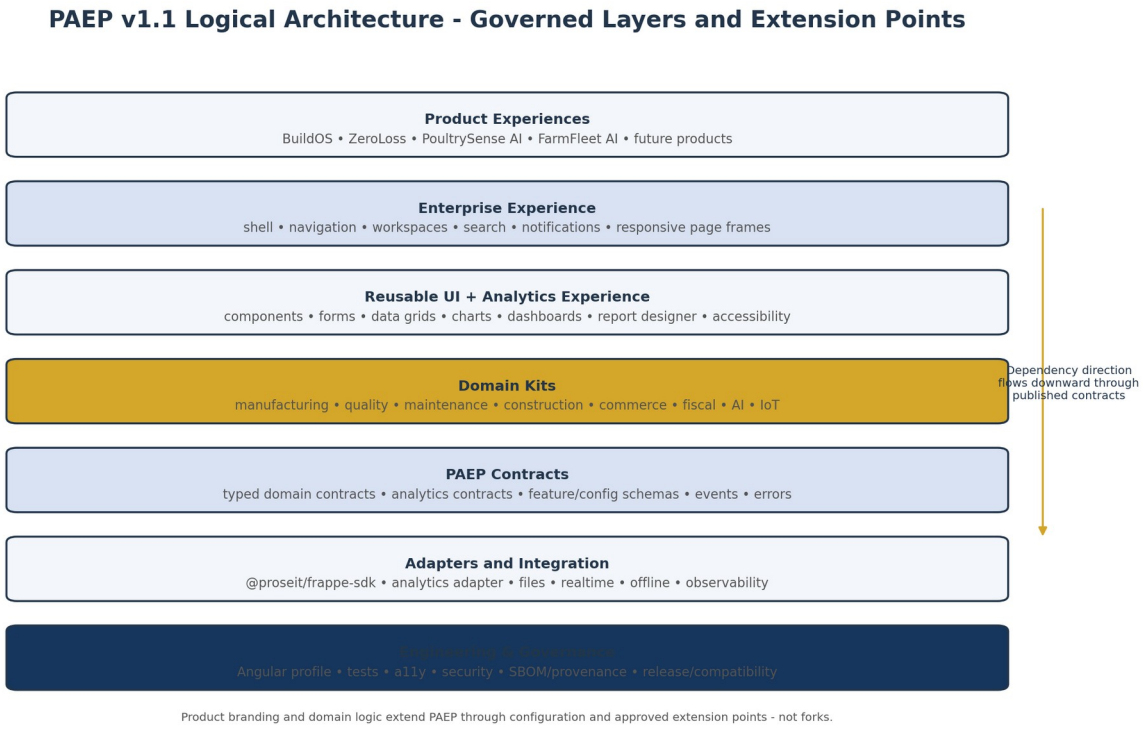


Figure 2A. PAEP v1.1 logical architecture with explicit contracts, analytics experience and governed dependency direction.

5.1 Recommended package boundaries

Package / area	Responsibility
@proseit/paep-foundation	Design tokens, typography, spacing, surfaces, elevation, motion, breakpoints, icons and global semantics.
@proseit/paep-components	Reusable accessible UI primitives and enterprise data-entry/presentation components.
@proseit/paep-enterprise	Application shell, navigation, search, notifications, command palette, workspace scaffolds and common page frames.
@proseit/paep-domain-*	Domain component kits that are reusable across more than one product or product family.
@proseit/frappe-sdk	Authentication/session, typed domain clients, workflows, files, lists, realtime, errors, idempotency/concurrency metadata and observability hooks.
@proseit/paep-offline	PWA service, queue/sync state, retry/reconciliation helpers and connectivity indicators.
@proseit/paep-i18n	Locale loading, translation dictionaries, number/date/currency formatting, RTL utilities and language switching.
@proseit/paep-observability	Frontend logs, metrics, tracing/correlation, error boundary and performance telemetry abstractions.
apps/*	Thin product applications that compose PAEP packages, route/product configuration and product-specific domain screens.

5.1.1 Additional Version 1.1 package boundaries

Package / area	Responsibility

@proseit/paep-contracts	Product-neutral typed domain, error, event, capability, configuration and analytics contracts. No Frappe implementation details.
@proseit/paep-analytics	Reusable report/dashboard shell, charts, scorecards, filters, drill-through, report parameters, saved views and export-job UX. Consumes reporting contracts only.
@proseit/paep-platform	Optional browser/platform abstractions for secure storage, connectivity, clipboard, download, viewport and environment capabilities when direct browser APIs would create portability or testability concerns.

5.2 Workspace rule

The preferred structure is a single governed Angular workspace/monorepo for the shared PAEP packages and reference applications, with product repositories consuming versioned packages or an approved source-sharing model. Teams shall avoid duplicating platform code into product repositories merely to accelerate local delivery.

5.3 Package dependency and public API rule

- Foundation and contracts packages shall not depend on enterprise, domain, adapter or product packages.
- Reusable components may depend on foundation/contracts but shall not import product-specific domain code.
- Domain and enterprise packages consume lower-level packages only through approved public entry points; deep imports into implementation folders are prohibited.
- Apps compose routes, configuration and product-specific workflows but shall not duplicate platform code. Architecture-boundary rules shall be checked in CI.
- Circular package dependencies are release-blocking unless a documented architecture exception and removal plan are approved.

5.4 Angular engineering profile

- New PAEP components, directives and pipes should be standalone-first unless a compatibility requirement justifies otherwise.
- UI state should use Angular-native reactive primitives for local and derived state; computed/derived state is preferred to duplicated mutable state. Global state libraries are introduced only where workflow complexity justifies them.
- PAEP releases shall be compatible with zoneless change detection where supported by the pinned Angular release; product code shall not rely on ZoneJS-specific behaviour without an approved exception.
- Route-level lazy loading and deferred loading shall be used for non-critical or heavy features when measurement shows benefit. Deferral shall preserve accessibility announcements and avoid harmful layout shift.
- Production builds shall use AOT compilation, strict TypeScript/template checking and approved lint/static-analysis rules.
- Direct DOM manipulation and security-bypass APIs are exceptional and require security review, documented rationale and focused tests.

6. Design Foundations and Tokens

PAEP design foundations provide visual consistency without forcing every product into one undifferentiated identity. Product branding is expressed through controlled token overrides, approved logos, domain imagery and navigation composition.

Foundation area	Control
Typography	Define a controlled type scale, weights, line heights and truncation rules; avoid ad-hoc per-screen font sizing.
Spacing	Use a consistent spacing scale for layout, component density and responsive adaptation.
Surfaces/elevation	Standardise page, panel, card, modal and raised-layer semantics; elevation communicates hierarchy rather than decoration.
Colour	Separate semantic colours (success/warning/error/info) from product accents; maintain contrast requirements.
Motion	Use bounded motion for feedback/state transitions; support reduced-motion preferences.
Responsive breakpoints	Define device/layout breakpoints and component adaptation rules; no screen should depend on fixed desktop width.

Foundation area	Control
Iconography	Use one governed icon vocabulary with accessible labels where meaning is not obvious.
Density	Support appropriate compact/comfortable density for data-heavy enterprise screens without sacrificing touch targets and accessibility.

6.1 Token governance

- Tokens are versioned assets and shall have release notes when semantic meaning changes.
- Product themes may override approved brand tokens but shall not override security/accessibility state semantics.
- Components consume semantic tokens, not raw colour/spacing literals, except inside controlled foundation code.
- Dark mode or additional themes may be introduced as platform capabilities, but must pass the same accessibility and regression gates.

6.2 Responsive, adaptive and interaction design profile

- Critical workflows shall preserve information and functionality at a narrow mobile viewport equivalent to 320 CSS pixels, except where a controlled two-dimensional presentation is essential.
- Data-heavy tables shall define an adaptive strategy such as priority columns, card/list transformation, frozen key columns, detail drill-down or bounded horizontal scrolling where two-dimensional comparison is essential.
- Primary actions shall remain discoverable without dependence on hover. Touch, keyboard, pointer and zoom/reflow interaction shall be verified for supported profiles.
- Layouts shall tolerate translation expansion, user font scaling and RTL without clipping essential actions or hiding status/context.
- PAEP UX patterns shall apply recognition over recall, progressive disclosure, meaningful defaults, error prevention, work preservation, consistent terminology and reversible actions where practical.
- Critical product workflows shall retain usability evidence using representative users or an approved equivalent evaluation, with task completion, error/recovery observations and material findings recorded.

7. Enterprise Application Shell

The shell establishes consistent navigation, identity context, product identity and cross-cutting actions while allowing product-specific information architecture.

Shell capability	Required behaviour
Topbar	Product/tenant context, global search/command access, notifications, help, locale and user actions as applicable.
Sidebar/navigation	Configuration-driven groups/routes, active-state clarity, collapsed/mobile behaviour and entitlement-aware explanatory states.
Workspace/page frame	Consistent title, context, actions, breadcrumbs when justified, status/filters and responsive content region.
Search/command palette	Keyboard-accessible global navigation and bounded actions; results respect backend permissions.
Notifications	Differentiate transient UI feedback from persistent domain notifications; deep links preserve tenant/product context.
Error boundary	Recover gracefully from frontend exceptions, retain correlation information and offer safe retry or navigation.
Locked/unavailable state	Explain why a feature is unavailable (permission, edition, configuration or environment) without implying client-side enforcement.
Offline indicator	Expose connectivity and synchronization state persistently enough for field users to understand data risk.

Additional shell capability	Required behaviour
Analytics/reporting workspace	Consistent dashboard/report frame, report title/context, date/tenant

	scope, filters, refresh/freshness state, export/share actions and drill-through context.
Saved views/preferences	User-scoped layout/filter/density preferences are persisted through approved storage/server profiles and never override permissions or authoritative report definitions.

8. Reusable Component Architecture

PAEP components are enterprise interaction contracts, not merely visual widgets. Each reusable component must define states, keyboard behaviour, semantics, loading/empty/error behaviour, responsive behaviour, data constraints and test evidence.

Component family	Representative components	Mandatory concerns
Actions	Button, split action, icon action, toolbar	Disabled/loading states, labels, keyboard, destructive confirmation.
Forms	Input, select, autocomplete, date/time, currency, quantity, file, rich text	Labels/help, validation, error association, locale formats, read-only, dirty state.
Data	Table/data grid, list, tree, cards, timeline	Pagination/virtualisation, sort/filter, selection, empty/loading/error, responsive alternatives.
Navigation	Tabs, menu, breadcrumb, stepper, pagination	Focus order, selected/current semantics, overflow and mobile behaviour.
Overlays	Dialog, drawer, popover, tooltip	Focus trap/restore, escape behaviour, accessible name/description, mobile sizing.
Feedback	Toast, inline alert, progress, skeleton, status badge	Persistent vs transient distinction, non-colour cues, screen-reader announcements.
Evidence	Attachment, image/evidence viewer, audit timeline, change history	Secure URLs, metadata, provenance, download/preview permissions.
Workflow	State badge, action panel, approval/rejection dialog, sign-off summary	Server-provided allowed actions, comments/reasons, confirmation and audit result.

8.1 State completeness rule

Every reusable screen-level pattern shall specify at minimum: loading, empty, populated, filtered/no-results, validation error, permission denied, edition unavailable, backend conflict, offline/queued (where applicable), retry/recovery and unexpected-error states.

8.2 Frontend state-management discipline

State category	PAEP rule
Component interaction state	Keep local to the owning component where practical; avoid application-wide stores for transient UI state.
Feature/workflow state	Owned by the feature/domain package with explicit lifecycle and typed interfaces.
Server state	Backend is authoritative; clients cache only under defined freshness/invalidation rules and reconcile conflicts explicitly.
Offline state	Stored only for enumerated offline workflows with tenant/user scope, lifecycle, encryption/protection where applicable and visible sync state.
Global/shared state	Limited to genuine cross-cutting context such as current identity, tenant, locale, feature capability and connectivity. New global mutable stores require architecture justification.

9. Domain Component Kits

Domain kits reduce duplicated interaction design where multiple products share a business concern. Domain kits shall remain reusable and shall not embed product-specific business rules that belong in product modules or backend services.

Domain kit	Representative reusable experience
Manufacturing	Work-order cards, operation confirmation, material issue/receipt, yield/scrap, machine status.
Quality	Specification/result capture, inspection grids, nonconformance/CAPA status and evidence.
Maintenance	Asset/functional-location selector, notification/order status, maintenance checklist and spare-parts usage.
Construction	Project/WBS context, RFI/submittal status, progress evidence, inspection/NCR and valuation summaries.
Commerce/payments	Catalogue/order status, payment intent/status, settlement/reconciliation presentation.
Fiscal	Fiscal document state, authority response/error, retry/reversal status - display only; fiscal authority remains backend.
AI	Grounded answer panel, source/provenance view, action proposal, approval gate, evaluation/trace status.
IoT/edge	Device status, telemetry tiles, alarm timeline, offline gateway health, sync/last-seen indicators.

10. PAEP Contracts, @proseit/frappe-sdk and Backend Integration

PAEP v1.1 separates product-neutral frontend contracts from backend implementation adapters. @proseit/paep-contracts defines stable typed interfaces and semantics consumed by PAEP components and product applications. @proseit/frappe-sdk remains the strategic ERPNext/Frappe adapter and implements those contracts against versioned PMRA/product APIs. This preserves the confirmed ERPNext/Frappe backend decision while reducing implementation leakage and improving testability, portability and acquisition/partnership flexibility.

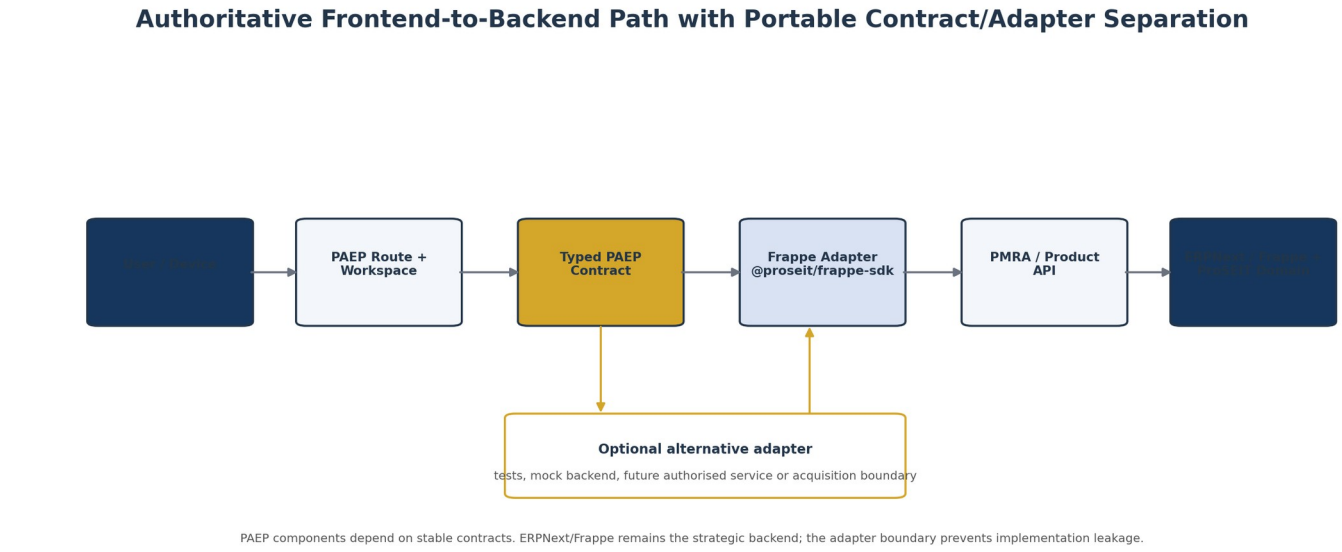


Figure 3A. Version 1.1 contract/adapter separation. ERPNext/Frappe remains strategic and authoritative; PAEP UI depends on product-neutral contracts rather than Frappe implementation details.

10.1 SDK capability contract

SDK capability	Required abstraction
Authentication/session	Login/logout, session renewal, current identity/tenant context and secure session error handling.
Typed domain clients	Versioned request/response models for PMRA/product APIs instead of arbitrary DocType access.
Lists/data grids	Pagination/cursor metadata, filters, sorting, export requests and server-side constraints.
Workflow	Retrieve state/allowed actions; submit, approve, reject, cancel or other governed transitions with reasons/evidence.
Files/evidence	Upload/download/preview metadata, secure access, checksum/provenance metadata where required.
Realtime	Subscription abstraction for permitted events, reconnect behaviour and stale-state recovery.
Errors	Consistent mapping for authentication, authorization, validation, conflict, rate limit and service failures; retain correlation IDs.
Idempotency/concurrency	Expose idempotency keys and server version/ETag metadata for operations that need retry or conflict protection.
Observability	Trace/correlation propagation, timing and failure telemetry without leaking sensitive payloads.

10.2 API discipline

- UI code shall not depend on database table names or implementation-private DocType fields unless an explicit contract designates them as stable platform metadata.
- Breaking API changes require versioning, compatibility tests and migration notes.
- Client-generated IDs may be used for offline-safe creates only where the domain contract explicitly supports them.
- Server validation errors should be structured so PAEP can associate errors to fields, records or global actions without parsing human prose.
- PAEP reusable UI packages shall depend on @proseit/paep-contracts, not on Frappe transport objects or DocType-specific implementation details.
- @proseit/frappe-sdk shall map PAEP contracts to approved PMRA/product APIs and shall be replaceable in tests by contract-conformant fakes/mocks without changing UI components.
- Contract changes and adapter changes are versioned independently where appropriate; compatibility is verified in CI.

11. Identity, Tenancy, Permissions and Edition Gating

PAEP presents access state; it does not become the security boundary. A hidden route or disabled button is never considered an authorization control.

Concern	Frontend rule
Tenant context	Every application shows or retains the active tenant/legal entity/project context required to prevent accidental cross-context work.
Permission state	Routes/actions may be hidden or disabled for usability, but every authoritative read/write remains server-authorized.
Edition state	Navigation and pages may show locked capabilities and upgrade messaging; resource limits and entitlements are server-enforced.
Segregation of duties	Approval screens reflect backend-derived allowed actions and must not infer approval authority solely from role names.
Session expiry	The UI must handle expired/invalid sessions without losing unsaved work when safe; reauthentication and retry rules are explicit.

Concern	Frontend rule
Impersonation/support	Any support impersonation capability must be backend-governed, prominently indicated and fully audited.
SSO/private deployment	Enterprise/Enterprise Plus identity providers are integrated through controlled authentication configuration, not product forks.

12. Routing, Runtime Configuration and Feature Composition

Product applications should be thin composition layers. Routes, product identity, supported locales, environment endpoints, feature flags and edition metadata are loaded from controlled configuration where practical.

Configuration area	Control
Product manifest	Product ID/name, logo/accent tokens, route groups, capability packages and build/release metadata.
Environment	API base, realtime endpoint, identity/SSO settings, file/CDN endpoints and telemetry endpoints; secrets are never embedded in public client configuration.
Capabilities	Feature flags or capability discovery reflect server-supported features; client flags must not grant authority.
Navigation	Route/menu metadata includes labels, icons, order, capability/edition hints and responsive behaviour.
Version compatibility	Application startup checks or telemetry must surface incompatible backend/SDK/platform versions before unsafe use.
Remote configuration	Only non-secret, explicitly approved configuration may be remotely mutable; high-risk behaviour changes require release/change control.

12.1 Runtime configuration schema and fail-safe startup

- Every runtime configuration document shall conform to a versioned typed schema and identify its schema version, product release and environment profile.
- Invalid, incomplete, unsigned/untrusted where signing is required, or incompatible configuration shall fail safely before protected product functionality is enabled.
- Runtime configuration may contain public endpoints, feature discovery, locale/product metadata and telemetry destinations; secrets and privileged credentials are prohibited.
- Where practical, the same compiled frontend artefact shall be deployable across approved environments by changing validated runtime configuration rather than rebuilding or forking source.
- Remote feature configuration shall support auditability, bounded rollout and rollback for high-impact changes.

13. Realtime, Files, Notifications and Background Work

Area	PAEP control
Realtime events	Treat realtime as an optimization for freshness, not as the sole source of truth. Reconnect must trigger safe refresh/reconciliation.
Long-running jobs	Display job identity, progress/status and final result; avoid holding a blocking browser request for server work that is asynchronous.
File upload	Show upload progress, failure/retry and server acceptance; preserve evidence metadata required by domain rules.
File preview/download	Use authorized URLs/tokens and content-type handling; do not expose private storage paths.

Area	PAEP control
Notifications	Persistent domain notifications are server-backed; transient toasts are UI feedback and must not substitute for durable evidence.
Exports	Large reports/exports should use background generation with expiry/authorization and clear user status.

14. Offline/PWA and Low-Bandwidth Architecture

PAEP is designed for African operating environments where field connectivity may be intermittent, expensive or constrained. Offline capability is therefore a controlled product feature rather than an informal browser cache.

State	Required user meaning
Online/current	The view is based on confirmed server state within the defined freshness policy.
Local draft	Data exists only on the device and has not been submitted.
Queued	The action is committed locally for synchronization but has not yet been accepted by the server.
Synced	The server accepted the action and returned authoritative identifiers/version.
Conflict	The server state changed or the operation cannot be safely replayed; user/domain resolution is required.
Failed	Synchronization failed and requires retry, correction or escalation; the UI must not imply success.

14.1 Offline controls

- Offline-enabled workflows must be explicitly enumerated; absence of network is not permission to bypass backend rules.
- Queued commands require client IDs/idempotency metadata where duplicate business effects are possible.
- Sensitive cached data shall be minimized, scoped, protected and expired according to security policy.
- The UI shall show pending synchronization before logout, device handover or other actions that risk losing local-only work.
- Conflict resolution must preserve both the attempted change and authoritative server state for user/domain reconciliation.
- Service-worker and cache upgrades shall not silently discard unsynchronized records.
- Offline and cached data shall be partitioned by tenant, user and product context where applicable; tenant switching, logout, account removal and device handover shall trigger the controlled retention/clear policy.
- Long-lived bearer credentials shall not be persisted in script-readable browser storage. Offline authentication/session continuity must follow the approved identity security profile.
- Synchronisation queues shall expose age/freshness and unresolved-item counts so users can judge operational risk.

15. Internationalisation and Localisation

PMRA-AF v2.0 requires internationalisation for English, French, Portuguese, Arabic/RTL, Amharic and extensible local-language dictionaries [1]. Product releases may activate languages according to validated market requirements, but the platform architecture shall remain locale-ready.

Concern	Rule
Text	No user-visible product text hardcoded inside reusable components except controlled fallback/developer diagnostics.
Formatting	Dates, times, numbers, percentages and currency use locale-aware formatting with explicit business timezone/currency context.
RTL	Layout, directional icons, tables and form flows are tested for RTL where Arabic is activated.

Concern	Rule
Translation keys	Stable semantic keys are versioned; product-specific strings stay outside shared packages unless truly reusable.
Long text	Layouts tolerate language expansion without clipping; translated strings are not forced into fixed-width controls.
Backend messages	Prefer structured error/status codes with localized frontend messages; do not depend on parsing English backend prose.

16. Accessibility and Inclusive Interaction

PAEP targets WCAG 2.2 Level AA for supported web experiences, as established by PMRA-AF v2.0 and BuildOS v2.0 [1], [2]. Accessibility is a platform quality attribute and component release gate.

Area	Minimum control
Keyboard	All interactive components are reachable and operable without a pointing device; focus order follows task flow.
Focus	Visible focus; dialogs/drawers manage focus entry, trap and restoration; route changes place focus predictably.
Semantics	Use native elements and appropriate ARIA only where needed; labels, names, roles and states are programmatically available.
Errors	Validation errors are associated with fields and summarized where needed; errors are not indicated by colour alone.
Contrast	Text, interactive states and essential graphics meet the controlled contrast target; product accents cannot override semantic accessibility.
Responsive/touch	Touch targets and spacing remain usable on mobile devices; zoom/reflow does not hide essential actions.
Status changes	Important asynchronous changes use appropriate live-region or equivalent notification patterns.
Manual evidence	Automated checks are supplemented by keyboard and assistive-technology/manual testing for critical workflows.

16.1 Additional WCAG 2.2 and usability controls

- Critical interfaces shall verify that keyboard focus is not obscured by sticky headers, overlays or persistent UI regions.
- Where an interaction uses dragging, an equivalent non-dragging method shall be provided unless dragging is essential.
- Interactive targets shall meet the controlled target-size profile, with documented exceptions only where WCAG permits.
- Help mechanisms, repeated data entry and authentication flows shall follow WCAG 2.2 requirements applicable to the product profile.
- Accessibility release evidence shall be complemented by usability evaluation for high-value or high-risk workflows; findings that materially affect completion, errors or recovery are tracked to resolution or controlled exception.

17. Security, Privacy and Frontend Trust Boundaries

FRONTEND TRUST BOUNDARY

Any data or code delivered to the browser must be treated as accessible to the user/device. Client-side hiding, obfuscation, local validation or source-map choices are not substitutes for backend authorization and server-side validation.

Control area	PAEP requirement
XSS/output handling	Prefer framework-safe templating; sanitize explicitly supported rich content; avoid unsafe HTML execution.
CSRF/session	Follow the backend authentication/session model; state-changing requests use the required anti-forgery/session protections.
Secrets	No server secrets, private keys or privileged credentials in browser bundles/runtime config.
Sensitive data	Minimize local/browser storage of personal, fiscal, credential or commercially sensitive data; define retention/clear rules.
Logging	Frontend logs/telemetry avoid raw sensitive payloads and include correlation metadata sufficient for diagnostics.
Dependencies	Approved dependency set, lockfiles, vulnerability review and SBOM are part of release evidence.
Content security	Deployment profiles should support a restrictive content-security posture and controlled third-party origins.
Uploads/downloads	Client file constraints improve UX but server validation, authorization and scanning remain authoritative.
Source maps	Production source-map exposure is controlled according to support/security policy; never assume source maps contain no sensitive information.

17.1 Mandatory Angular/browser security profile

- Production PAEP deployments shall use Angular AOT compilation. Runtime/JIT template generation from untrusted data is prohibited.
- Production deployment profiles shall enforce a restrictive Content Security Policy (CSP) appropriate to required origins; weakening the policy requires security approval and evidence.
- Trusted Types enforcement shall be enabled where supported by the approved browser/deployment profile. Use of Angular security-bypass APIs is exceptional and reviewed.
- Direct DOM APIs and third-party DOM-manipulation libraries shall be avoided where Angular templates/platform APIs provide a safe alternative.
- Long-lived bearer tokens, private keys and equivalent privileged credentials shall not be stored in localStorage, IndexedDB or other script-readable persistence. Session/storage design follows the controlled identity profile.
- Browser caches, offline stores and downloadable report artifacts containing sensitive information shall have explicit retention, tenant/user scoping and clearing rules.

18. Performance, Reliability and Observability

PAEP performance targets must be defined in a product Quality Attribute Contract rather than as generic claims. The platform supplies common budgets, instrumentation and test hooks; each product validates representative workloads.

Concern	Platform control / evidence
Initial load	Route-level lazy loading, asset budgets, compression/caching policy and measured representative-device performance.
Data-heavy screens	Server pagination/filtering, virtualization where justified, bounded DOM size and cancellation of obsolete requests.
Network resilience	Timeout/retry policy differentiated by safe/idempotent vs unsafe operations; clear user state during degradation.
Frontend errors	Global error boundary/reporting with release version, route, correlation and environment context.
Tracing	Propagate correlation/trace identifiers into backend calls and retain them in surfaced support diagnostics.

Concern	Platform control / evidence
Core web/runtime metrics	Collect relevant browser performance metrics without unnecessary surveillance or sensitive data.
Compatibility	Supported browser/device matrix is controlled per PAEP release and verified before product release.
Recovery	Reload/reconnect does not duplicate business actions; unsaved/local queue state is preserved according to offline policy.

18.1 Reporting and dashboard performance controls

- Interactive dashboards shall use bounded datasets, server-side aggregation and progressive/lazy loading rather than transferring unbounded fact data to the browser.
- Visual queries shall be cancellable when filters or drill context change; stale responses shall not overwrite newer user intent.
- Dashboard/report freshness shall be visible where data is cached, asynchronously refreshed or drawn from a curated analytical store.
- Report export and large analytical workloads shall use background jobs with status, expiry and authorization rather than long-running browser requests.
- Product QACs shall define representative dashboard/report timing targets, maximum query/output sizes and concurrency assumptions.

19. Clean-Room Provenance and Licence Governance

The clean-room objective is architectural ownership and freedom from proprietary commercial-template runtime/redistribution constraints - not an assertion that the software has no licences. PAEP must retain evidence showing how designs and implementations were produced and what third-party obligations remain [1].

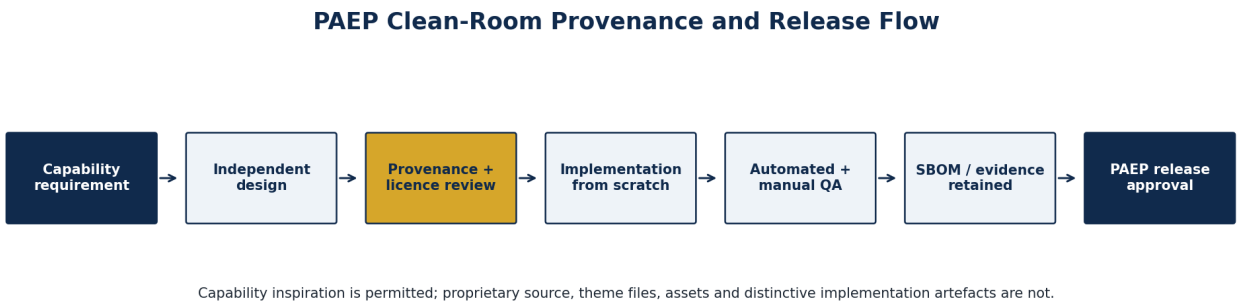


Figure 4. PAEP clean-room provenance and release flow.

Control	Required evidence
Requirements/capability source	Documented business/UX need expressed as capabilities and interaction requirements rather than copied pixel/source implementation.
Design provenance	Design rationale, sketches/prompts/specifications and reviewer record sufficient to show independent creation.
Prohibited inputs	No copied proprietary source, theme files, assets or distinctive implementation artefacts from commercial templates.
Open-source dependencies	Package inventory, licence, version, rationale and approval; obligations reflected in distribution/documentation.
Generated design/code assistance	Prompts/instructions request capabilities and product-specific design, not replication of proprietary screens; generated output is reviewed before adoption.

Control	Required evidence
SBOM	Release-level software bill of materials retained in an approved machine-readable format where supported.
Attestation	Release checklist confirms provenance/licence review, test evidence and unresolved exceptions before approval.

20. Testing, Quality Attribute Contracts and Release Gates

Test layer	Minimum coverage
Unit	Token utilities, state functions, SDK mapping, validators and logic with deterministic unit tests.
Component	Rendering, states, keyboard/focus, events, validation and responsive variants in isolation.
Accessibility	Automated checks plus manual keyboard/focus and assistive-technology testing for critical components/workflows.
Visual regression	Controlled screenshots for reusable components, shell states and representative breakpoints; approved changes are intentional.
Integration	SDK with backend/API contracts, workflows, files, realtime, authentication/session and representative error paths.
Contract	Generated/typed client models and backend schemas validated for compatibility; breaking changes fail the gate.
Offline/sync	Queue, retry, duplicate prevention, conflict, service-worker upgrade and recovery fault-injection tests.
Security	Dependency scanning, frontend security tests, authorization-negative integration scenarios and deployment configuration checks.
Performance	Bundle/asset budget plus representative device/network and data-volume scenarios against product QAC targets.
End-to-end	Golden product workflows across route, permissions, backend state, notifications/evidence and error/recovery paths.

20.1 Platform Quality Attribute Contract baseline

ID	Concern	Example target form	Verification
PAEP-QAC-ACC-01	Accessibility	WCAG 2.2 AA target; controlled exception register	Automated + manual a11y evidence
PAEP-QAC-PERF-01	Performance	Named bundle/route budget + p75/p95 user-facing timing on representative device/network	Bundle analysis + browser performance test
PAEP-QAC-SEC-01	Authorization presentation	Zero cases where client gating is accepted as the sole authorization control	Negative integration/security tests
PAEP-QAC-OFF-01	Offline integrity	Zero duplicate business effects in defined retry/replay suite; all unresolved conflicts surfaced	Offline fault-injection/reconciliation test
PAEP-QAC-UPG-01	Upgradeability	No unresolved critical regression across supported product compatibility matrix	Compatibility + migration test
PAEP-QAC-I18N-01	Localisation	No clipped critical UI in activated locales/RTL; locale-aware formats verified	Visual/i18n test suite

ID	Concern	Example target form	Verification
PAEP-QAC-UX-01	Usability	Critical workflows complete without unresolved severity-1 usability findings; material error/recovery observations recorded.	Representative-user task evaluation / approved expert evaluation
PAEP-QAC-CFG-01	Configuration integrity	Invalid/incompatible runtime configuration fails safely; no secrets in public configuration.	Schema validation + negative startup tests
PAEP-QAC-PORT-01	Deployment portability	Approved build runs in defined cloud/private/on-prem profiles without source fork; environment differences supplied through controlled runtime configuration where practical.	Deployment matrix test
PAEP-QAC-RPT-01	Reporting responsiveness	Representative dashboard/filter/drill interactions meet product-defined p75/p95 targets with bounded datasets.	Analytics workload test
PAEP-QAC-RPT-02	Metric governance	Controlled measures produce consistent results across dashboard, detail and export for the same filter/security context.	Semantic model/contract tests

21. PAEP Showcase / UI Laboratory and Developer Experience

PAEP shall maintain a browsable showcase/UI laboratory containing component examples, states, responsive variants, accessibility notes and usage guidance. This is both engineering documentation and evidence that reusable components are defined independently of any single product.

Developer asset	Purpose
Component showcase/UI lab	Live examples of primitives, enterprise patterns, domain kits, states and responsive/accessibility behaviour.
Design token catalogue	Semantic token names, values, product override rules and deprecation history.
SDK examples	Authentication, list/detail, workflow, files, errors, realtime, offline and correlation patterns.
Starter application	Reference product shell demonstrating route composition, tenant context, i18n, editions and error/offline states.
Architecture decision records	Record high-impact frontend decisions, dependency changes and exceptions.
Migration guides	Actionable upgrade notes for PAEP package/API changes and deprecated components.
Testing utilities	Shared mocks, test harnesses, accessibility helpers, contract-test fixtures and visual-regression tooling.

Additional developer asset	Purpose
Analytics/reporting laboratory	Reference dashboards, chart states, filters, drill-through, empty/error/freshness states, export jobs and responsive report layouts using synthetic data.
Contract/adapters test harness	Verifies UI components against PAEP contracts independently of Frappe implementation and verifies @proseit/frappe-sdk conformance against backend contracts.
Runtime-config validator	Developer/CI tooling for schema validation, compatibility checks and fail-safe configuration tests.

22. Versioning, Packaging and Release Management

Rule	PAEP position
Release unit	PAEP release has a controlled version, release notes, dependency/SBOM evidence, compatibility matrix and migration guidance.
Angular version	Framework version is pinned in the PAEP release manifest and upgrade policy rather than hardcoded as a permanent architecture rule.
Semantic changes	Breaking public component/SDK contracts require a major release or approved compatibility layer/migration window.
Deprecation	Deprecated APIs/components remain documented for a defined migration period; removal requires explicit release notes.
Package publication	Only approved packages and entry points are public; internal implementation paths are not treated as stable contracts.
Product compatibility	Each product declares supported PAEP and backend contract versions; CI validates the intended matrix.
Rollback	Release process retains the ability to revert the frontend bundle/configuration and coordinates with backend compatibility where necessary.

22.1 Deployment portability profile

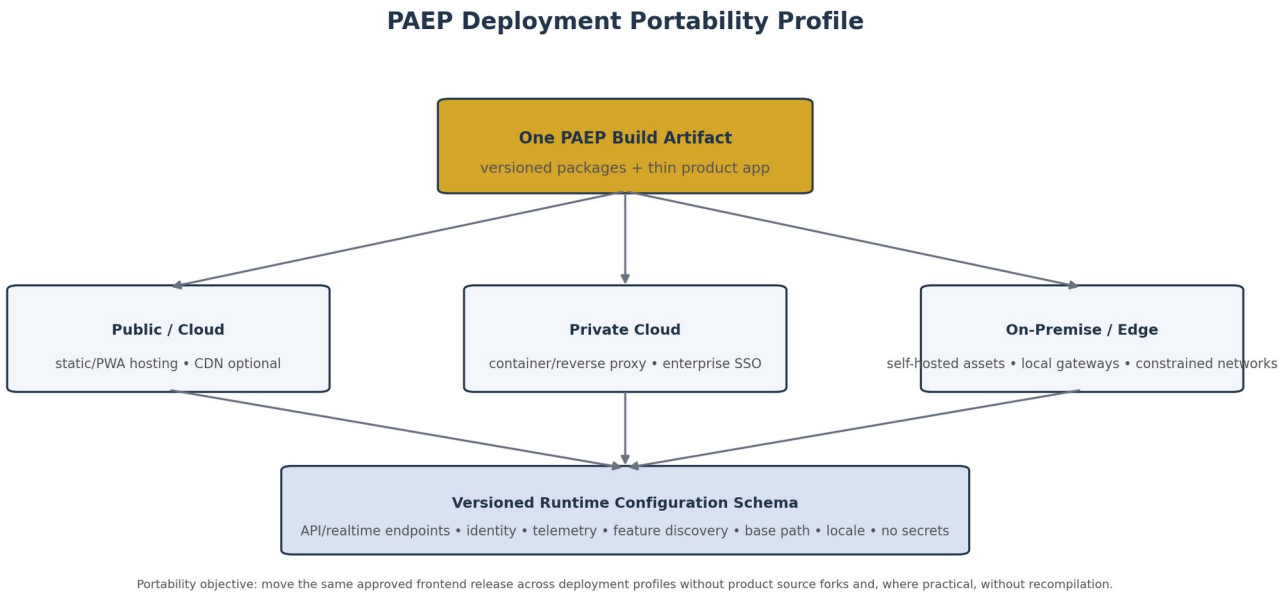


Figure 7. PAEP deployment portability profile. The objective is one controlled release deployable across approved hosting models through validated runtime configuration, not environment-specific source forks.

- PAEP shall support standards-compliant static/PWA hosting and may be packaged behind a reverse proxy or container for private/on-premise deployment.
- Deployment shall support configurable base paths/subpaths, runtime-resolved service endpoints and self-hosted assets where required.
- CDNs, SaaS telemetry and third-party hosted assets are optional deployment capabilities rather than baseline runtime dependencies.
- Product releases declare supported browser/device and deployment profiles in the compatibility matrix.
- SSR/hydration may be enabled for public or content-oriented experiences where justified; authenticated transactional ERP experiences are not required to adopt SSR.

23. Product Adoption and Migration

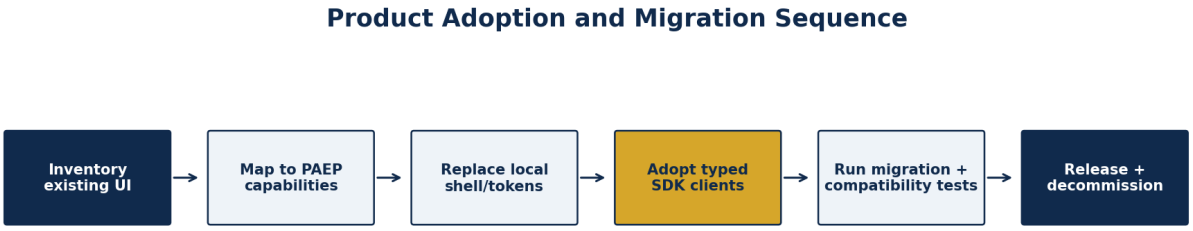


Figure 5. Product adoption and migration sequence.

Existing product frontends should migrate incrementally. The objective is to remove duplicated shell/design/integration code while preserving validated product workflows and avoiding a disruptive “big-bang” rewrite.

Step	Required output
1. Inventory	Routes, screens, local components, direct backend coupling, licences, tokens/themes, accessibility/offline gaps and technical debt.
2. Capability mapping	Map each UI element to PAEP foundation/component/enterprise/domain/SDK or justified product-specific ownership.
3. Shell/token adoption	Replace local shells and uncontrolled design values with PAEP shell/tokens while preserving product identity.
4. Contract migration	Replace arbitrary DocType/direct transport coupling with approved typed domain clients and error contracts.
5. State completion	Add loading/empty/error/permission/edition/offline/conflict/recovery states required by PAEP.
6. Evidence tests	Run accessibility, visual, integration, offline, performance and regression tests.
7. Release/decommission	Release by bounded capability; remove duplicate code only after evidence-backed cutover and rollback readiness.

24. Implementation Roadmap

Phase	Primary deliverables	Exit evidence
P0 - Governance baseline	Repository rules, clean-room evidence register, dependency approval, release/version policy	Architecture approval; provenance process operational
P1 - Foundation	Tokens, typography, responsive grid, icons, base semantics	Token showcase; visual/a11y tests
P2 - Core components	Forms, actions, data display, overlays, feedback, navigation	Component lab + component/a11y/visual tests
P3 - Enterprise shell	Shell, navigation, workspace frame, search/command, notification, error/offline indicators	Reference application passes responsive/a11y/error-state tests
P4 - Frappe SDK	Auth/session, typed clients, workflow, files, lists, errors, realtime, observability	Contract/integration test evidence against reference backend

Phase	Primary deliverables	Exit evidence
P5 - Offline/PWA+i18n	Queue/sync, PWA update, locale/RTL foundations	Offline fault-injection + locale/RTL evidence
P6 - Domain kits	Manufacturing/quality/maintenance/ construction/AI/IoT kits according to reuse demand	Cross-product usage evidence; no duplicated local implementation
P7 - Product migration	Golden vertical slice then incremental portfolio migration	Compatibility matrix, regression/UAT and decommission evidence
P6A - Reporting & analytics	Analytics UI kit, semantic/report contracts, reporting service integration, export jobs and representative dashboards.	Operational + analytical path evidence; security/tenant tests; performance QAC; dashboard/export UX evidence.

24.1 Golden vertical slice

The first formal PAEP acceptance should prove one real end-to-end product workflow rather than only a component library. The slice should include authentication, tenant/edition context, navigation, a data-heavy screen, form validation, a governed workflow action, file/evidence handling, backend errors/conflict, responsive behaviour, accessibility, observability and at least one offline/low-bandwidth scenario where the selected product requires it.

25. Architecture Acceptance and Evidence

Gate	Minimum evidence
Architecture	Package boundaries, public contracts, dependency map, runtime configuration, ADRs and approved exceptions.
Clean-room/IP	Design provenance, prohibited-input attestation, dependency licence register and SBOM.
Component quality	Showcase examples, unit/component tests, visual regression and accessibility evidence.
Integration	Typed API/SDK contract tests, workflow/files/realtime/error scenarios and backend compatibility evidence.
Security/privacy	Threat model contribution, sensitive-data/local-storage review, dependency security and negative authorization tests.
Offline/PWA	Defined offline-enabled workflows, queue/conflict/retry/service-worker upgrade evidence.
Internationalisation	Activated locale/RTL checks, formatting and layout expansion evidence.
Performance	Bundle/route budgets and representative workload/device/network results.
Product release	Compatibility matrix, migration/rollback notes, UAT/regression, Product Owner and Solutions Architect acceptance.

Additional gate	Minimum evidence
Angular engineering	Strict build/lint evidence, package-boundary checks, AOT production build and zoneless compatibility evidence for the pinned Angular release where applicable.
Configuration/portability	Versioned runtime-config schema, fail-safe tests and approved deployment-profile matrix without uncontrolled source forks.
Reporting/analytics	Semantic metric definitions, tenant/row security tests, representative dashboards, export/background-job tests, operational-load isolation evidence and performance/usability results.
UX	Responsive/adaptive evidence plus representative-user or approved equivalent usability evaluation for critical workflows.

26. Principal Risks and Controls

Risk	Consequence	Control
Clean-room provenance not demonstrable	IP/licensing dispute or weak acquisition diligence	Evidence register, dependency review, prohibited-input rule, release attestation and retained design rationale.
Platform becomes a visual theme only	Products still duplicate integration/state/quality logic	Treat SDK, shell, offline, i18n, a11y and testing as first-class PAEP packages.
Over-centralisation	Product teams blocked by platform backlog	Define stable extension points/domain kits and publish release/compatibility policy.
Frontend trust leakage	Unauthorized or invalid operations	Backend authority, negative tests, typed contracts and no client-only permission/edition enforcement.
Direct Frappe coupling returns	Upgrade fragility and duplicated validation	SDK/domain API contract rule plus architecture review/CI linting where practical.
Offline duplication/conflict	Duplicate transactions or silent data divergence	Idempotency, client IDs, explicit states, conflict UI and fault-injection tests.
Accessibility regression	Exclusion and procurement/compliance risk	Component-level a11y DoD, automated+manual gates and controlled exception register.
Dependency sprawl	Security/licence/bundle risk	Approved dependency catalogue, SBOM, version policy and periodic rationalisation.
Product forks	High maintenance cost and divergence	Configuration/token/domain-extension model; forks require architecture exception and sunset plan.
Framework upgrade churn	Unplanned migration effort	Pin versions per PAEP release, compatibility matrix, scheduled upgrade cadence and migration guides.

Additional risk	Consequence	Control
State-management sprawl	Inconsistent behaviour, duplicated cache truth and difficult upgrades	State locality rules, authoritative server state, approved shared-state patterns and architecture review for global stores.
Configuration drift	Environment-specific failures or unsafe behaviour	Versioned schemas, startup validation, compatibility checks, controlled remote changes and deployment matrix.
Analytics overloads OLTP	Slow ERP transactions and unstable dashboards	Reporting service boundary, bounded operational queries, aggregation/cache and optional curated analytics store for heavy history.
Metric inconsistency	Different dashboards report conflicting KPIs	Governed semantic measures/dimensions, versioned definitions, contract tests and controlled metric ownership.
BI vendor lock-in	Reduced portability or acquisition flexibility	PAEP analytics contracts and visualization/reporting adapters; vendor embedding remains optional, not baseline.

27. Rich Reporting and Analytics Architecture

PAEP shall provide a reporting and analytics experience capable of executive dashboards, operational monitoring, scorecards, trend analysis, drill-through, cross-filtering, saved views and governed exports at a level suitable for enterprise products. The objective is Power BI-class interaction quality without making Power BI or any single BI vendor a mandatory runtime dependency.

The reporting architecture preserves the original PAEP decisions: ERPNext/Frappe remains the authoritative transactional backend; browser clients consume typed contracts rather than database models; permissions and tenancy remain server-authoritative; product applications remain thin composition layers; and heavy analytical workloads may be separated from operational transaction processing.

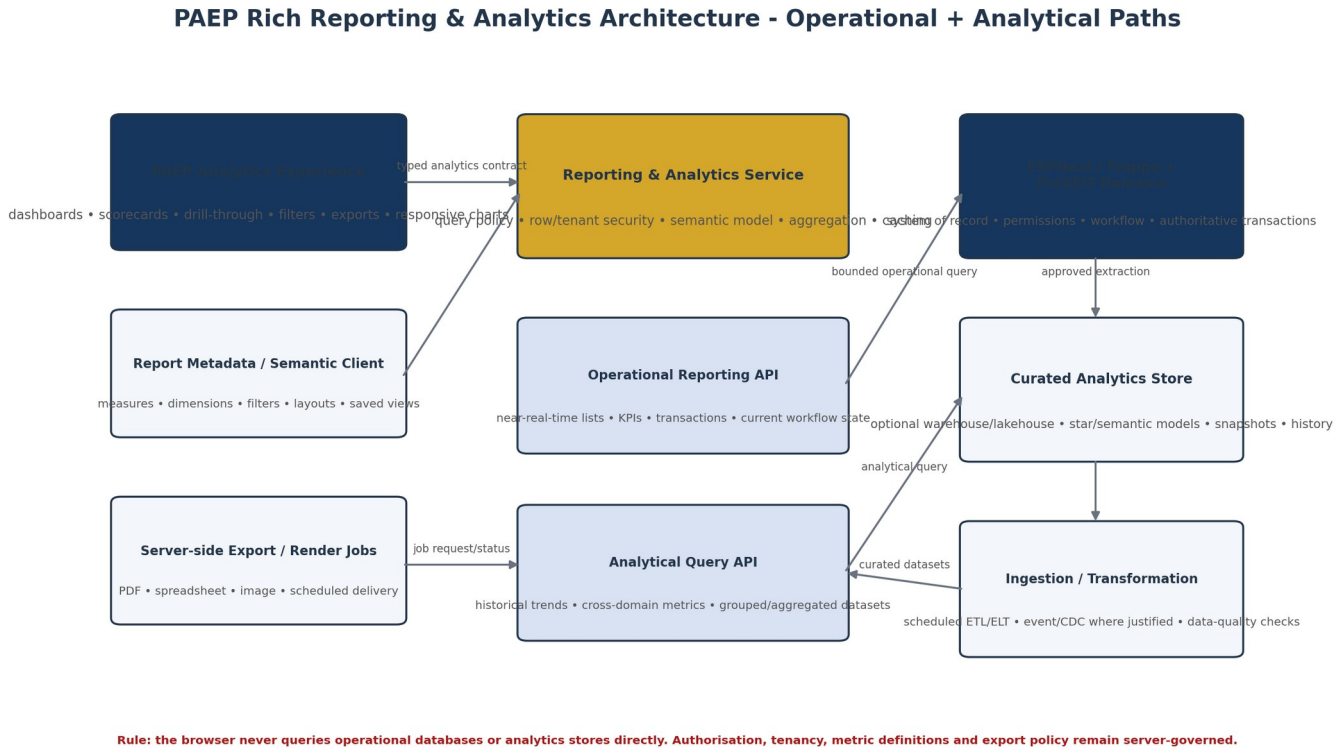


Figure 6. PAEP rich reporting and analytics architecture. Operational and analytical paths are separated while sharing governed semantic definitions and authorization.

27.1 Reporting capability model

Capability	PAEP requirement
Dashboards and scorecards	Responsive multi-visual layouts, KPI cards, trends, variance, targets, status, annotations and configurable refresh/freshness indicators.
Interactive analysis	Cross-filtering, drill-down, drill-through, date/period selection, dimension slicing, search, sorting and contextual detail.
Report parameters	Typed filter/parameter definitions with validation, defaults, allowed values, dependent filters and backend-enforced scope.
Saved views	User/team saved filters, column/layout preferences and favourites with explicit ownership and sharing rules.
Tabular analytics	Enterprise grids with server-side paging/grouping/aggregation, pinned key fields, totals/subtotals, conditional presentation and accessible mobile alternatives.
Exports	Governed spreadsheet/PDF/image/data exports through background jobs for large outputs; export permissions may be stricter than screen-view permissions.
Scheduled delivery	Optional server-side scheduling/subscriptions with recipient authorization, expiry and audit evidence. Browser timers are not authoritative scheduling.
Explainability	Metric definitions, units, source/freshness, filters and calculation notes are discoverable so decision-makers can interpret results.

27.2 Semantic reporting contract

- A report shall be defined against governed measures, dimensions, hierarchies, filters and datasets rather than arbitrary SQL or browser-side aggregation over raw operational tables.
- Metric definitions shall have stable identifiers, owner, version, unit, aggregation rule, applicable dimensions, security scope and calculation/source description.
- The same semantic definition shall be reused across dashboard visual, tabular detail and export where the business meaning is intended to be identical.
- Report metadata and semantic definitions may be product-specific or cross-product, but shared definitions are promoted to controlled PAEP/PMRA contracts only when genuinely reusable.
- Row-level/tenant security and permitted dimensions/filters are enforced server-side. Frontend filter controls are usability affordances, not data-security controls.

27.3 Operational versus analytical reporting

Reporting path	Use	Architecture rule
Operational reporting	Current work orders, inventory positions, workflow queues, exceptions, machine state, recent transactions and near-real-time operational KPIs.	Query approved domain/reporting APIs against authoritative services using bounded filters, pagination and aggregation. Do not run unrestricted analytical queries over transactional persistence from the browser.
Analytical reporting	Long-period trends, cohort/benchmark analysis, multi-site comparisons, historical snapshots, cross-domain measures and large aggregations.	Use the Reporting & Analytics Service and, where workload justifies it, a curated analytics store populated through approved ETL/ELT or event/CDC pipelines.
Hybrid dashboard	Executive view combining current operational signals with historical context.	Each visual declares source/freshness; semantic definitions reconcile meaning across operational and analytical sources.

27.4 Analytics store decision rule

A separate analytics store is not mandatory for every product or report. Begin with governed operational reporting APIs. Introduce a curated warehouse/lakehouse or equivalent analytical store when one or more of the following drivers are evidenced: long historical retention, expensive multi-dimensional aggregation, cross-product/site analytics, reporting concurrency that threatens transactional performance, snapshot/time-series requirements, or materially different data-retention/governance needs.

- ERPNext/Frappe and product domain services remain systems of record; the analytical store is derived and rebuildable according to its data contract.
- Ingestion pipelines record source, extraction time, transformation version, quality checks and reconciliation status for controlled metrics.
- Analytical datasets shall be tenant-isolated/logically secured according to the product security model and shall not weaken authoritative permissions.
- Freshness/service-level targets are product-specific and visible to users where stale data could affect decisions.

27.5 Visualization and report rendering

- @proseit/paep-analytics provides product-neutral chart, KPI, filter, report-frame and drill interaction contracts. Specific charting/grid libraries are replaceable implementation dependencies selected through dependency/licence/security review.
- Visualizations shall use semantic design tokens, accessible labels/alternatives, keyboard-operable controls where interactive, non-colour cues and responsive degradation for narrow screens.
- Complex visuals shall provide an accessible tabular or textual equivalent where necessary for critical information.
- Large exports and pixel-perfect documents are rendered server-side or by background workers; the browser coordinates requests and presents status/download results.
- Embedding an external BI product, including Power BI, may be supported through an approved analytics adapter for customers who require it, but embedded BI shall not become a mandatory PAEP runtime dependency or bypass PAEP/server authorization.

27.6 Reporting security, privacy and governance

- Report/query authorization is evaluated on every server request using current identity, tenant, project/legal-entity scope, permissions and applicable edition/capability.
- Export, share, schedule and subscription rights are independently governable from screen-view rights.
- Sensitive fields are excluded or masked by server policy; client-side hiding is insufficient.
- Query audit records should capture report/dataset identifier, principal, tenant context, time, high-level filter scope, result status and correlation ID without unnecessarily logging sensitive payloads.
- Caching keys shall include all security-relevant context; shared caches must not permit cross-tenant or cross-permission leakage.

27.7 Reporting UX and quality targets

- Dashboards shall communicate loading, partial, stale, empty, permission-limited, failed and refreshed states explicitly.
- Filters shall expose active scope clearly and provide predictable reset/apply behaviour; drill-through shall preserve navigational context and support return to the originating analysis.
- Dense executive dashboards shall prioritize decision relevance over visual decoration and avoid redundant visual encodings.
- Report designers/configurators, where offered, shall be permissioned capabilities using governed datasets and semantic fields rather than arbitrary database access.
- Product QACs define dashboard interaction latency, data freshness, export turnaround, supported data volumes, accessibility and usability evidence.

Appendix A. PAEP Repository Reference Structure

```
paep/
├── apps/
│   ├── reference-app/
│   └── ui-lab/
├── packages/
│   ├── foundation/
│   ├── contracts/
│   ├── components/
│   ├── enterprise/
│   ├── analytics/
│   ├── platform/
│   ├── domain-manufacturing/
│   ├── domain-quality/
```

```
│   ├── domain-maintenance/
│   ├── domain-construction/
│   ├── domain-commerce/
│   ├── domain-fiscal/
│   ├── domain-ai/
│   ├── domain-iot/
│   ├── frappe-sdk/
│   ├── offline/
│   ├── i18n/
│   └── observability/
├── tooling/
│   ├── architecture-boundaries/
│   ├── config-validation/
│   ├── test-utils/
│   ├── visual-regression/
│   ├── analytics-test/
│   ├── a11y/
│   └── build-release/
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── reporting-metrics/
│   ├── provenance/
│   ├── licences/
│   └── migration/
└── evidence/
    ├── tests/
    ├── accessibility/
    ├── usability/
    ├── analytics/
    ├── performance/
    ├── portability/
    ├── security/
    ├── sbom/
    └── releases/
```

TAILORING RULE

The folder names are a reference structure, not a mandate to use an exact monorepo tool. Any alternative shall preserve the same ownership boundaries, public-contract discipline, evidence, provenance and release controls.

Appendix B. PAEP Clean-Room Definition of Done

- Component/screen requirements and design rationale are documented.
- No proprietary template source, assets, theme files or copied distinctive implementation artefacts are incorporated.
- Design provenance and approved external dependencies are recorded.
- Accessibility tests target WCAG 2.2 Level AA for supported web experiences.
- Responsive, keyboard and focus behaviour is verified.
- Frappe/API contracts are consumed through approved SDK/domain clients.
- Loading, empty, error, permission, edition, offline/conflict and recovery states are implemented where applicable.
- Unit/component, integration, visual regression and product-relevant performance tests pass.
- SBOM and licence/provenance review are retained.
- Component documentation and examples are available in the PAEP showcase/UI laboratory.
- Breaking changes include compatibility assessment, migration guidance and release notes.
- Product Owner and Solutions Architect approval evidence is retained for controlled releases.
- Package dependency-direction and public-entry-point checks pass.
- Production build uses AOT and the approved CSP/Trusted Types deployment profile is verified where applicable.

- Runtime configuration conforms to the controlled schema and fail-safe negative tests pass.
- Responsive/adaptive behaviour and critical workflow usability evidence are retained.
- Reporting components use governed analytics contracts and do not query databases/analytics stores directly.
- Approved deployment-profile portability evidence is retained for releases that claim multiple deployment profiles.

Appendix C. Component Acceptance Checklist

Check	Acceptance question
Purpose	Is the component reusable across at least one defined PAEP/product context and is its task/semantic role clear?
API	Are inputs/outputs/public APIs minimal, typed, documented and stable?
States	Are loading/empty/error/disabled/read-only/permission/offline/conflict states defined as applicable?
Keyboard/focus	Can the complete interaction be performed and exited using keyboard with predictable focus?
Semantics	Are label, name, role, value/state and status updates programmatically available?
Responsive	Does it work at required breakpoints and touch sizes without clipping or hidden critical actions?
i18n/RTL	Are strings externalized; locale formatting/long text/RTL tested where applicable?
Security/privacy	Does it avoid embedding secrets, exposing unauthorized data or logging sensitive payloads?
Performance	Is rendering/data-volume behaviour bounded and measured for the expected use?
Tests/evidence	Are component, a11y, visual and relevant integration tests retained and green?
Provenance	Are design/source provenance and dependency licences recorded?
Docs	Is the component represented in the UI lab with examples and usage guidance?

Appendix D. Product Adoption Checklist

- Product UI inventory and architecture gap assessment completed.
- Product routes and capability ownership mapped to PAEP vs product-specific code.
- Local tokens/shell replaced or reconciled with PAEP; branding retained through approved overrides.
- Direct/uncontrolled backend coupling replaced with approved SDK/domain clients.
- Permissions/edition UI is explanatory only and backend enforcement verified.
- Activated locales, responsive breakpoints and accessibility tested.
- Offline-enabled workflows explicitly identified and sync/conflict behaviour verified.
- Performance/bundle budgets and observability enabled.
- Compatibility matrix, migration/rollback notes and product UAT/regression evidence retained.
- Duplicate legacy platform code removed only after successful cutover.
- Product state ownership and caching strategy mapped to PAEP state-management rules.
- Product runtime configuration migrated to the versioned PAEP schema and tested for fail-safe startup.
- Reporting inventory classifies operational versus analytical workloads; metrics/dimensions are mapped to governed reporting contracts.
- Heavy historical/cross-domain reports have documented analytical-store decision and OLTP isolation evidence where applicable.
- Approved cloud/private/on-premise deployment profiles are tested without product source forks.

Appendix E. Acronyms

Acronym	Meaning
ADR	Architecture Decision Record
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
ASR	Architecturally Significant Requirement
CDK	Component Development Kit
CI/CD	Continuous Integration / Continuous Delivery
CSP	Content Security Policy
ETag	HTTP Entity Tag / version validator
i18n	Internationalisation
IP	Intellectual Property
LTR	Left-to-Right
PAEP	ProSEIT Angular Experience Platform
PMRA-AF	ProSEIT Manufacturing Reference Architecture for Africa
PWA	Progressive Web Application
QAC	Quality Attribute Contract
RBAC	Role-Based Access Control
RTL	Right-to-Left
SBOM	Software Bill of Materials
SDK	Software Development Kit
SSO	Single Sign-On
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
CDC	Change Data Capture
ELT	Extract, Load, Transform
ETL	Extract, Transform, Load
OLTP	Online Transaction Processing

Appendix F. References

[1] ProSEIT Organisation of Software Professionals Ltd, ProSEIT Manufacturing Reference Architecture for Africa (PMRA-AF), Version 2.0, PROSEIT-STD-PMRA-AF-001, 25 Aug. 2026.

[2] ProSEIT Organisation of Software Professionals Ltd, ProSEIT Built Environment - Construction Industry Opportunity & Product Architecture (BuildOS), Version 2.0 review candidate, PROSEIT-STD-BUILDOS-AF-001, 3 Sep. 2026.

[3] H. Washizaki, Ed., Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0, IEEE Computer Society, 2024.

[4] ISO/IEC/IEEE 42010:2022, Software, systems and enterprise - Architecture description.

[5] ISO/IEC/IEEE 12207:2026, Systems and software engineering - Software life cycle processes.

[6] ISO/IEC/IEEE 29148:2018, Systems and software engineering - Life cycle processes - Requirements engineering.

[7] ISO/IEC 25010:2023, Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model.

[8] W3C, Web Content Accessibility Guidelines (WCAG) 2.2.

[9] OWASP, Application Security Verification Standard (ASVS), used by PMRA-AF as a software-security assurance reference.

[10] SPDX, software package data / software bill of materials standard referenced by PMRA-AF for dependency metadata and provenance.

[11] Angular, Security best practices, Angular documentation: AOT production compilation, Content Security Policy, Trusted Types and safe DOM handling.

[12] Angular, Deferred loading with @defer, Angular documentation: code splitting, loading/error states and accessibility considerations.

[13] Angular, Zoneless, Angular documentation: zoneless change detection and compatibility guidance for current Angular releases.

REFERENCE CONTROL

This child specification inherits the normative software-engineering, security and platform references of PMRA-AF Version 2.0. Product teams shall use the controlled PMRA standards register for exact applicable editions, profiles and evidence requirements rather than treating this appendix as an exhaustive standards catalogue.

Appendix G. Angular Engineering Profile

Area	Controlled profile
Composition	Standalone-first components/directives/pipes for new PAEP code unless compatibility justifies otherwise; thin applications compose versioned packages.
Reactivity/state	Angular-native signals/reactive primitives for local/derived UI state where appropriate; state locality and server authority rules apply.
Change detection	Zoneless-compatible code for supported Angular releases; dependencies that require ZoneJS are identified and controlled.
Loading	Route lazy loading and @defer for heavy/non-critical features when justified by performance evidence; accessible loading/error announcements.
Compilation	AOT production builds, strict TypeScript/template checking, approved lint/static analysis and package-boundary enforcement.
DOM/security	Prefer Angular templates/platform APIs; direct DOM access and bypassSecurityTrust* use are exceptional, reviewed and tested.
Testing	Unit/component/integration/e2e plus a11y, responsive, visual and performance evidence for reusable platform capabilities.

Appendix H. Reporting and Analytics Acceptance Checklist

Check	Acceptance question
Business meaning	Are measures, dimensions, units, filters and freshness understandable and governed?
Security	Are tenant/row/field permissions enforced server-side for view, drill, export, share and schedule actions?
Source path	Is the report correctly classified as operational, analytical or hybrid, with no browser-to-database/store access?
Performance	Are datasets bounded and representative filter/drill/dashboard timings within the product QAC?
Consistency	Do dashboard, detail and export produce semantically consistent results for the same context?
Freshness	Is data age/freshness visible where caching, ETL or asynchronous

	refresh could affect decisions?
Responsive UX	Do visuals, filters and tables adapt to supported mobile/tablet/desktop widths without loss of critical function?
Accessibility	Are key insights available without colour alone and through accessible names/tabular or textual alternatives where required?
Exports/jobs	Are large exports/background jobs authorized, auditable, status-visible and expiry-controlled?
Portability	Can visualization/reporting implementation be changed through adapters without changing product business semantics?
Evidence	Are usability, performance, security, metric-contract and regression results retained?